Resumo – Recursividade

1. Antes de tudo: a pilha de chamadas

Para entender recursão, primeiro é preciso entender o que acontece quando **uma função chama outra**. Comece pelo exercício que abre a aula: o que é impresso, e em que ordem?

```python
def primeiro():
    print("primeiro: inicio")
    segundo()
    print("primeiro: fim")


def segundo():
    print("segundo: inicio e fim")


print("main: inicio")
primeiro()
print("main: fim")
```

```
main: inicio
primeiro: inicio
segundo: inicio e fim
primeiro: fim
main: fim
```

Repare que `primeiro: fim` só aparece depois que `segundo` termina. Isso acontece porque, quando uma função chama outra:

- a função que chamou **para e fica esperando**;
- o computador guarda onde ela parou em um **quadro**;
- os quadros ficam **empilhados**: quem executa está sempre no **topo**;
- quando a função termina, seu quadro sai e a de baixo **continua de onde parou**.

Acompanhe a pilha durante a execução acima (em cada linha, o último nome é o topo):

momento	pilha (base → topo)	o que aconteceu
1	main	imprime “main: inicio”
2	main → primeiro	**empilha**; imprime “primeiro: inicio”
3	main → primeiro → segundo	**empilha**; imprime “segundo: inicio e fim”
4	main → primeiro	**desempilha**; primeiro continua e imprime “primeiro: fim”
5	main	**desempilha**; main continua e imprime “main: fim”

Macete

A última chamada a entrar é a primeira a sair, exatamente como uma pilha de pratos: você só tira o prato de cima. Toda questão de recursão é, no fundo, uma questão de acompanhar essa pilha.

2. O que é uma função recursiva

Imagine uma pilha de caixas. Dentro de cada caixa pode existir uma chave ou outras caixas menores, que também podem conter caixas. Para procurar a chave, fazemos sempre a **mesma ação**: abrir a caixa atual; se encontrar a chave, parar; se encontrar outra caixa, procurar a chave dentro da nova caixa.

Uma **função recursiva** é uma função que **chama ela mesma** para resolver uma **versão menor** do mesmo problema. Para o computador, não há nada de novo: é uma função chamando uma função, e cada chamada ganha o seu próprio quadro na pilha.

```python
def eh_caixa(item):
    return type(item) == list


def procurar_chave(caixa):
    for item in caixa:
        if item == "chave":
            return True
        if eh_caixa(item):
            if procurar_chave(item):
                return True
    return False


caixa = ["meia", ["livro", ["papel"]], [["chave"]]]
print(procurar_chave(caixa))
print(procurar_chave(["meia", ["livro"]]))
```

```
True
False
```

Aqui as caixas foram representadas por listas, e `eh_caixa` apenas verifica se o item é uma lista. O ponto central é a linha `procurar_chave(item)`: para resolver o problema da caixa grande, usamos **o mesmo procedimento** em uma caixa menor.

Atenção

Esse exemplo usa `for` porque cada caixa tem vários itens. Nas listas de exercícios, porém, o professor costuma pedir que a solução recursiva **não use estruturas de repetição**. Nesses casos, a recursão é que faz o papel do laço.

3. Caso base e caso recursivo

Toda função recursiva precisa das duas partes:

Caso base	Caso recursivo
encerra a recursão	a função chama ela mesma
responde **sem chamar de novo**	sempre com um **problema menor**
é o problema "tão pequeno" que a resposta é imediata	reduz o problema e confia que a chamada menor devolve a resposta certa

Macete

Para montar qualquer função recursiva, responda às três perguntas da conclusão da aula, nesta ordem: **(1)** qual é o caso base? **(2)** o problema diminui a cada chamada? **(3)** o que acontece na ida e o que acontece na volta? Se não souber responder a primeira, ainda não é hora de escrever código.

4. Exemplo completo: fatorial

O fatorial de n, escrito $n!$, é o produto de todos os inteiros de 1 até n. Por exemplo, $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$.

Passo 1 — encontrar a recursão. Escreva alguns fatoriais um embaixo do outro:

$$\begin{aligned} 5! &= 5 \times \underbrace{4 \times 3 \times 2 \times 1}_{4!} \\ 4! &= 4 \times \underbrace{3 \times 2 \times 1}_{3!} \\ 1! &= 1 \end{aligned}$$

Cada fatorial contém o fatorial do número anterior: $5! = 5 \times 4!$, $4! = 4 \times 3!$ e assim por diante, até $1! = 1$, que tem resposta imediata.

Passo 2 — escrever a definição recursiva.

$$n! = \begin{cases} 1 & \text{se } n = 1 \quad \text{(caso base)} \\ n \times (n-1)! & \text{se } n > 1 \quad \text{(caso recursivo)} \end{cases}$$

Lê-se: se n for igual a 1, devolvemos 1. Caso contrário, devolvemos n vezes o fatorial de $n - 1$.

Passo 3 — transformar cada linha da definição em código.

```python
def fatorial(n):
    # caso base
    if n == 1:
        return 1
    # caso recursivo
    else:
        return n * fatorial(n - 1)


print(fatorial(1))
print(fatorial(3))
print(fatorial(5))
print(fatorial(6))
```

```
1
6
120
720
```

Macete

A definição matemática **é** o código. Cada linha da chave vira um ramo do `if`: o caso base vira o `if` com `return` direto, e o caso recursivo vira o `else` com a chamada. Sempre escreva a definição primeiro: a tradução para Python fica quase automática.

5. Ida e volta: o que acontece na execução

Acompanhe `fatorial(3)` quadro a quadro.

Ida (empilhando). Nenhuma conta é feita: cada chamada descobre que precisa de outra e fica esperando.

chamada	n	situação
`fatorial(3)`	3	n não é 1: precisa de `fatorial(2)`, espera `3 * fatorial(2)`
`fatorial(2)`	2	n não é 1: precisa de `fatorial(1)`, espera `2 * fatorial(1)`
`fatorial(1)`	1	**caso base**: devolve 1 sem chamar ninguém

Volta (desempilhando). Cada chamada recebe o resultado da de cima e faz a **sua** multiplicação.

chamada	o que faz
`fatorial(1)`	devolve 1
`fatorial(2)`	recebeu 1: devolve $2 \times 1 = 2$
`fatorial(3)`	recebeu 2: devolve $3 \times 2 = 6$
`main`	recebeu 6 e imprime

Macete

O modelo mental da aula em uma frase: **na ida, nenhuma conta é feita: cada chamada espera a próxima. Na volta, cada chamada recebe um resultado e faz a sua parte.** Para resolver no papel, desenhe uma "escada": desça escrevendo as chamadas até o caso base, depois suba escrevendo os resultados.

6. E se faltar o caso base?

```
def contagem(n):
    print(n)
    contagem(n - 1)


contagem(3)
```

```
3 2 1 0 -1 -2 -3 -4 ...
RecursionError: maximum recursion depth exceeded
```

Essa função **nunca para** de chamar a si mesma: cada chamada empilha um novo quadro, nenhum quadro é retirado e a pilha cresce até estourar. O Python interrompe o programa com `RecursionError`.

Atenção

Ter um caso base não basta: ele precisa ser **alcançável**. A função `fatorial` acima, chamada com `fatorial(0)`, passa direto pelo `n == 1` e desce para -1, -2, ... até estourar. Sempre confira: **partindo de qualquer entrada válida, as chamadas chegam ao caso base?** Quando houver dúvida, use um caso base mais "largo", como `n <= 1` em vez de `n == 1`.

7. Praticando o método: soma_ate

Problema (exercício da aula). Escreva uma função recursiva `soma_ate(n)` que devolva a soma de todos os números de 1 até n. Por exemplo, soma_ate(4) $= 4 + 3 + 2 + 1 = 10$.

Definição recursiva. A soma até 4 é 4 mais a soma até 3:

$$\text{soma}(n) = \begin{cases} 1 & \text{se } n = 1 \quad \text{(caso base)} \\ n + \text{soma}(n-1) & \text{se } n > 1 \quad \text{(caso recursivo)} \end{cases}$$

```python
def soma_ate(n):
    if n == 1:
        return 1
    else:
        return n + soma_ate(n - 1)


print(soma_ate(1))
print(soma_ate(3))
print(soma_ate(4))
print(soma_ate(10))
```

```
1
6
10
55
```

Pilha de `soma_ate(3)`: na ida, `soma_ate(3)` espera `3 + soma_ate(2)`, que espera `2 + soma_ate(1)`, que é o caso base e devolve 1. Na volta: $2 + 1 = 3$, depois $3 + 3 = 6$.

Macete

Compare `fatorial` e `soma_ate`: a estrutura é **idêntica**, só muda a operação (`*` ou `+`). Muitos problemas são variações desse mesmo molde: "resposta de n = algo com n + resposta de $n - 1$".

8. Antes e depois da chamada recursiva

Este é o ponto que mais aparece nas questões de leitura de código. Descubra o que é impresso:

```python
def mostrar(x):
    print(x)
    if x > 0:
        mostrar(x - 1)
    print(x)


mostrar(2)
```

```
2
1
0
0
1
2
```

chamada	saída
`mostrar(0)`	0 / 0
`mostrar(1)`	1 / 0 / 0 / 1
`mostrar(2)`	2 / 1 / 0 / 0 / 1 / 2
`mostrar(3)`	3 / 2 / 1 / 0 / 0 / 1 / 2 / 3

A saída é sempre um **espelho**. O `print` que está **antes** da chamada executa na **ida** (2, 1, 0). O `print` que está **depois** da chamada só executa quando ela termina, ou seja, na **volta** (0, 1, 2).

Macete

Antes da chamada = ida = ordem em que as chamadas acontecem. Depois da chamada = volta = ordem inversa. Quer imprimir de 1 até n com uma função que diminui n? Coloque o `print` **depois** da chamada. Quer imprimir de n até 1? Coloque **antes**.

Atenção

O caso base de `mostrar` é "implícito": quando `x` vale 0, o `if` é falso, nenhuma chamada nova acontece e a função termina. Isso também é um caso base válido, mesmo sem `return`.

9. Versão iterativa versus recursiva

O mesmo problema escrito das duas formas:

```
def contar_ate_zero_while(n):
    while n >= 0:
        print(n)
        n = n - 1


def contar_ate_zero(n):
    if n < 0:
        return
    print(n)
    contar_ate_zero(n - 1)


contar_ate_zero_while(2)
contar_ate_zero(2)
```

```
2
1
0
2
1
```

```
0
```

	com while	com recursão
quando parar	condição do laço: `n >= 0`	caso base: `if n < 0: return`
como o valor diminui	`n = n - 1`	a chamada recebe `n - 1`

Macete

Repare que a condição se **inverte**: o `while` diz quando **continuar** (`n >= 0`), e o caso base diz quando **parar** (`n < 0`). Para converter um laço em recursão: a condição de parada vira o caso base, o corpo do laço vira o código da função e a atualização da variável vira o argumento da chamada.

Recursão é sempre melhor?

Não. Cada chamada ocupa um quadro na pilha, então a recursão gasta mais memória que um laço, existe um limite de chamadas empilhadas e, para contagens simples, o laço costuma ser mais eficiente. Por outro lado, a recursão é mais clara em problemas naturalmente recursivos, como o das caixas. **Escolha pela clareza, sabendo o preço da pilha.**

10. Dois casos base e duas chamadas: Fibonacci

Na sequência de Fibonacci, cada termo é a soma dos dois anteriores: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

$$\text{fib}(n) = \begin{cases} 0 & \text{se } n = 0 \\ 1 & \text{se } n = 1 \\ \text{fib}(n-1) + \text{fib}(n-2) & \text{se } n > 1 \end{cases}$$

```python
def fib(n):
    # casos base
    if n == 0:
        return 0
    elif n == 1:
        return 1
    # caso recursivo: duas chamadas
    else:
        return fib(n - 1) + fib(n - 2)


print(fib(4))
print(fib(9))
```

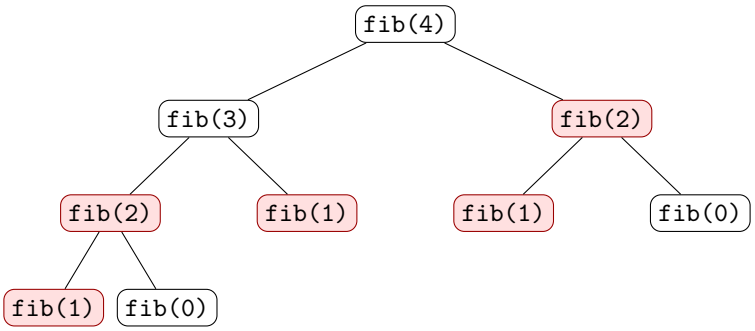
```
3
34
```

Atenção

Por que dois casos base? O caso recursivo usa $n-1$ **e** $n-2$. Com só `n == 1` como base, `fib(2)` chamaria `fib(0)`, que chamaria `fib(-1)`, e a pilha estouraria. Regra prática: se a chamada recua até k passos, você precisa de k casos base.

A árvore de chamadas

Como cada chamada abre **duas** novas chamadas, a execução não forma mais uma fila, e sim uma **árvore**:



Em vermelho, as chamadas que o computador refaz do zero: `fib(2)` é calculado duas vezes e `fib(1)` três vezes. Quanto maior o n, mais trabalho repetido: para calcular `fib(40)` são mais de 330 milhões de chamadas. **A recursão resolve, mas não controla o custo.**

Macete

Para descobrir o valor de `fib(n)` na prova sem desenhar a árvore inteira, escreva a sequência a partir do zero (0, 1, 1, 2, 3, 5, 8, ...) e conte as posições começando em 0. Para **contar quantas chamadas** acontecem, aí sim desenhe a árvore e conte os nós: `fib(4)` faz 9 chamadas.

11. Erros mais comuns

Atenção

- **Esquecer o caso base**, ou colocar um caso base que nunca é alcançado.
- **Não diminuir o problema:** chamar `fatorial(n)` dentro de `fatorial(n)` em vez de `fatorial(n - 1)`.
- **Esquecer o return na chamada recursiva:** escrever `n * fatorial(n - 1)` sem `return` calcula o valor e o joga fora; a função devolve `None`.
- **Confundir ida e volta:** achar que o `print` que está depois da chamada é executado logo em seguida. Ele espera todas as chamadas de cima terminarem.
- **Usar laço quando a questão proíbe:** se o enunciado diz "sem estruturas de repetição", a recursão é o laço.
- **Caso base de string ou número errado:** para textos, o caso base costuma ser a string vazia (`texto == ""`); para algarismos, um número com um único dígito (`n < 10`).

12. Questões de exemplo resolvidas

Os quatro exercícios de fixação da aula, com o raciocínio completo.

Questão 1 — imprimir_crescente(n)

Imprima os números de 1 até n, um por linha.
Raciocínio. Caso base: $n = 0$, não há nada a imprimir. O problema diminui chamando com $n - 1$. Como o valor **diminui** mas a impressão deve **crescer**, o `print` vai **depois** da chamada, para executar na volta.

```
def imprimir_crescente(n):
    if n == 0:
        return
    imprimir_crescente(n - 1)
    print(n)


imprimir_crescente(3)
```

```
1
2
3
```

Na ida, empilham-se as chamadas com 3, 2, 1 e 0 sem imprimir nada. A chamada com 0 retorna; então a com 1 imprime 1, a com 2 imprime 2 e a com 3 imprime 3.

Questão 2 — potencia(base, expoente)

Calcule a base elevada ao expoente usando só multiplicações.
Raciocínio. $2^5 = 2 \times 2^4$. Qualquer número elevado a 0 vale 1: esse é o caso base. Só o expoente diminui; a base é repassada igual.

$$\text{potencia}(b, e) = \begin{cases} 1 & \text{se } e = 0 \\ b \times \text{potencia}(b, e-1) & \text{se } e > 0 \end{cases}$$

```
def potencia(base, expoente):
    if expoente == 0:
        return 1
    else:
        return base * potencia(base, expoente - 1)


print(potencia(2, 5))
print(potencia(7, 0))
```

```
32
1
```

Questão 3 — contar_digitos(n)

Diga quantos dígitos tem o inteiro positivo n.
Raciocínio. Um número menor que 10 tem um único dígito: caso base. Para os demais, `n // 10` "corta" o último dígito, produzindo um problema menor. A resposta é 1 (o dígito cortado) mais os dígitos do restante.

```
def contar_digitos(n):
    if n < 10:
```

```
        return 1
    else:
        return 1 + contar_digitos(n // 10)


print(contar_digitos(5027))
print(contar_digitos(8))
```

```
4
1
```

Pilha de `contar_digitos(5027)`: chama com 502, depois 50, depois 5 (caso base, devolve 1). Na volta: $1 + 1 = 2$, $1 + 2 = 3$, $1 + 3 = 4$.

Macete

Em problemas com algarismos, use a dupla: `n % 10` dá o **último** algarismo e `n // 10` dá o **número sem ele**. Em problemas com texto, use a dupla: `texto[0]` dá o **primeiro** caractere e `texto[1:]` dá o **texto sem ele**.

Questão 4 — inverter(texto)

Devolva o texto de trás para frente.
Raciocínio. A string vazia invertida é ela mesma: caso base. Para inverter *casa*, inverta *asa* (que dá *asa*) e coloque o *c* no **final**: *asac*. O `texto[1:]` é o texto sem o primeiro caractere: um problema menor a cada chamada.

```
def inverter(texto):
    if texto == "":
        return ""
    else:
        return inverter(texto[1:]) + texto[0]


print(inverter("recursao"))
print(inverter("casa"))
```

```
oasrucer
asac
```

Questão 5 — Leitura de código

Escreva a saída do programa abaixo.

```
def calcular(n):
    print("Entrada", n)

    if n <= 1:
        print("Base", n)
        return n

    resultado = n + calcular(n - 2)

    print("Retorno", n, resultado)
    return resultado
```

```
print("Resultado", calcular(6))
```

Raciocínio. O problema diminui de 2 em 2. As mensagens "Entrada" saem na **ida**; as mensagens "Retorno" saem na **volta**, na ordem inversa.

chamada	ida	espera	volta
`calcular(6)`	Entrada 6	$6 + \text{calcular}(4)$	$6 + 6 = 12$
`calcular(4)`	Entrada 4	$4 + \text{calcular}(2)$	$4 + 2 = 6$
`calcular(2)`	Entrada 2	$2 + \text{calcular}(0)$	$2 + 0 = 2$
`calcular(0)`	Entrada 0	caso base	devolve 0

```
Entrada 6
Entrada 4
Entrada 2
Entrada 0
Base 0
Retorno 2 2
Retorno 4 6
Retorno 6 12
Resultado 12
```

13. Recapitulando

1. Uma função recursiva chama ela mesma para um problema menor.
2. O caso base encerra; o caso recursivo continua.
3. Cada chamada vira um quadro na pilha: a última a entrar é a primeira a sair.
4. Na ida as chamadas empilham; na volta os resultados voltam.
5. O que está antes da chamada executa na ida; o que está depois executa na volta.
6. Sem caso base alcançável, a pilha cresce até estourar.

14. Cola rápida

Tipo de problema	Molde
acumular até n (soma, produto)	`if n == 1: return 1` / `return n * f(n - 1)`
imprimir de n até 1	`print(n)` **antes** de `f(n - 1)`
imprimir de 1 até n	`print(n)` **depois** de `f(n - 1)`
algarismos de um número	base `n < 10`; usar `n % 10` e `f(n // 10)`
caracteres de um texto	base `texto == ""`; usar `texto[0]` e `f(texto[1:])`
percorrer lista por índice	base `i == len(lista)`; chamar `f(lista, i + 1)`
duas extremidades	base `inicio >= fim`; chamar `f(t, inicio + 1, fim - 1)`
recua 2 passos (Fibonacci)	dois casos base; `f(n - 1) + f(n - 2)`